

# Performance Tuning

# Outline

---

- Tuning Performance
- Suggested reading
  - 5.14

# Program Example

---

## Task

Analyzing the n-gram statistics of a text document  
an n-gram is a sequence of n words occurring in a document

reads a text file,

creates a table of unique n-grams

specifying how many times each one occurs

sorts the n-grams in descending order of occurrence

# Program Example

---

## Steps

- Convert strings to lowercase

- Apply hash function

- Read n-grams and insert into hash table

  - Mostly list operations

  - Maintain counter for each unique n-gram

- Sort results

## Data Set

- Collected works of Shakespeare

- 965,028 total words, 23,706 unique

- $N=2$ , called bigrams

- 363,039 unique bigrams

# Amdahl's Law

---

$$S = T_{\text{old}} / T_{\text{new}} = 1 / [(1-\alpha) + \alpha/k]$$

$$S_{\infty} = 1 / (1-\alpha)$$

$$T_{\text{old}} = 209.0$$

$$\alpha = T_{\text{part}} / T_{\text{old}} = 203.7 / 209.0 = 0.974$$

$$S_{\infty} = 1 / (1-\alpha) = 39.0$$

$$T_{\text{old}} / T_{\text{new}} = 209.0 / 5.4 = 38.5$$

- Try to optimize
  - the most time consuming part of a program
  - hottest part

# Identify the hottest part

---

- Using a very useful method profiling
  - Compile the program correctly
  - Run it with **typical input data**
  - Collect information from the result
  - **Analysis the result**
    - How much time did execution of a function take?
    - How many times was a particular function called?

# Using GPROF

---

## Step 1: Instrument the program

```
$gcc -Og -pg prog.c -o prog
```

Adds profiling code to `prog`, that is "Instruments" `prog.c`

## Step 2: Run the program

```
$/prog file.txt
```

Creates file `gmon.out` containing statistics

## Step 3: Create a report

```
$gprof prog > myreport
```

Uses `prog` and `gmon.out` to create textual report

## Step 4: Examine the report

```
$cat myreport
```

# Examples

| %     | cumulative | self    |          | self   | total  |              |
|-------|------------|---------|----------|--------|--------|--------------|
| time  | seconds    | seconds | calls    | s/call | s/call | name         |
| 97.58 | 203.66     | 203.66  | 1        | 203.66 | 203.66 | sort_words   |
| 2.32  | 208.50     | 4.85    | 965027   | 0.00   | 0.00   | find_ele_rec |
| 0.14  | 208.81     | 0.30    | 12511031 | 0.00   | 0.00   | Strlen       |

## Each line describes one function

**name:** name of the function

**%time:** percentage of time spent executing this function

**cumulative seconds:** [skipping, as this isn't all that useful]

**self seconds:** time spent executing this function

**calls:** number of times function was called (excluding recursive)

**self s/call:** average time per execution (excluding descendents)

**total s/call:** average time per execution (including descendents)

By default, the timings for library functions are not shown



# How does gprof work?

---

Essentially, by randomly sampling the code as it runs  
... and seeing what line is running, & what function it's in

## Interval counting

- Maintain a counter for each function

- Record the time spent executing this function

- Interrupted at regular time (1ms~10ms)

- Check which function is executing when interrupt occurs

- Increment the counter for this function

The calling information is quite reliable

# Performance Tuning

---

- Steps to improve execution (time) efficiency:
  - (1) Do timing studies
  - (2) Identify hot spots

calling history in the report

| %     | cumulative | self    |          | self   | total  |              |
|-------|------------|---------|----------|--------|--------|--------------|
| time  | seconds    | seconds | calls    | s/call | s/call | name         |
| 97.58 | 203.66     | 203.66  | 1        | 203.66 | 203.66 | sort_words   |
| 2.32  | 208.50     | 4.85    | 965027   | 0.00   | 0.00   | find_ele_rec |
| 0.14  | 208.81     | 0.30    | 12511031 | 0.00   | 0.00   | Strlen       |

# Performance Tuning

---

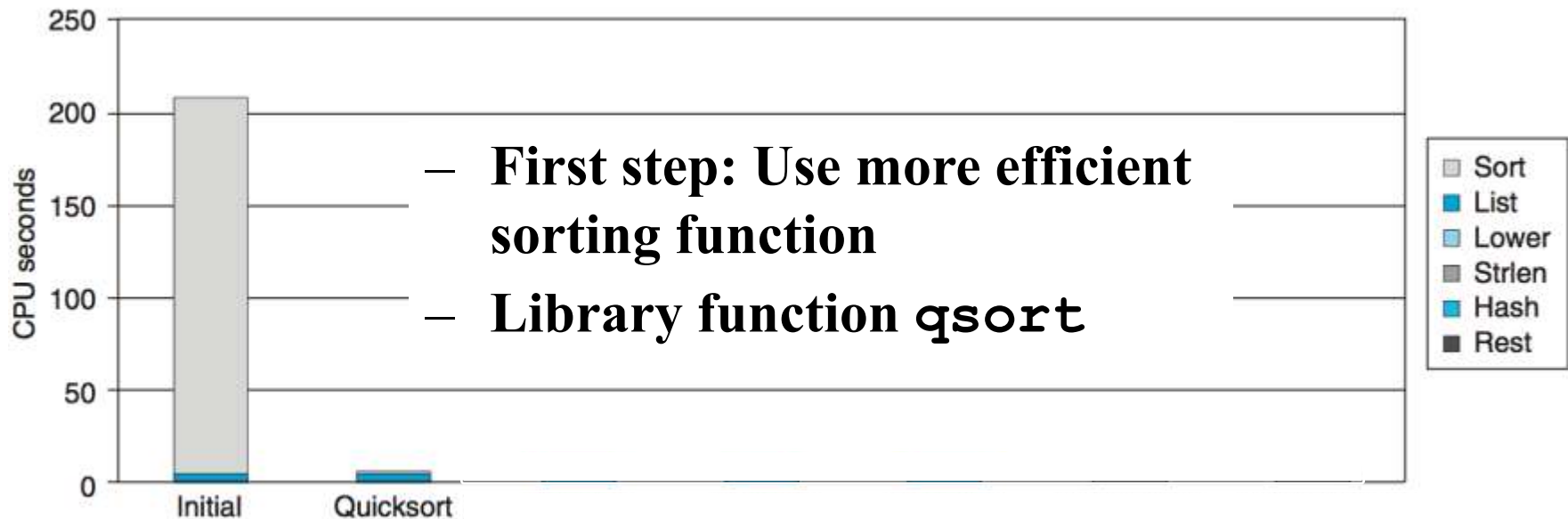
- Steps to improve execution (time) efficiency:
  - (1) Do timing studies
  - (2) Identify hot spots



# Use a Better Algorithm

---

- Steps to improve execution (time) efficiency:
  - (1) Do timing studies
  - (2) Identify hot spots
  - (3) Use a better algorithm or data structure
  - (4) Tune the code



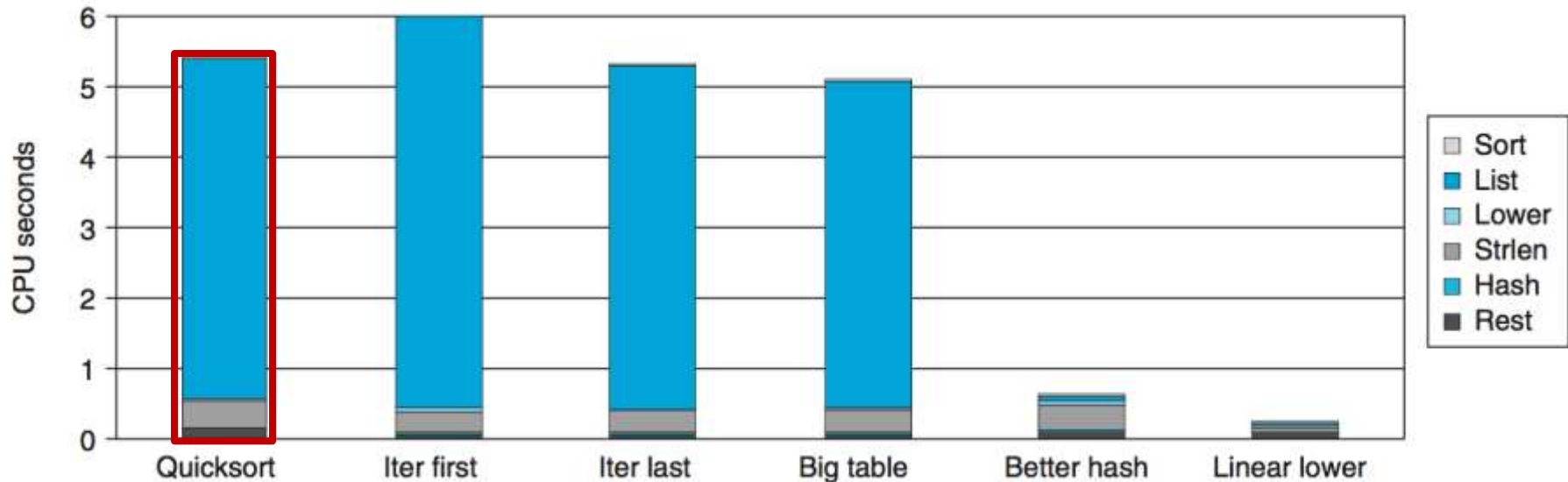
# Performance Tuning

---

| index | %   | time | self | called           | name              |
|-------|-----|------|------|------------------|-------------------|
|       |     |      |      | 158655725        | find_ele_rec [5]  |
|       |     | 4.85 | 0.10 | 965027/965027    | insert_string [4] |
| [5]   | 2.4 | 4.85 | 0.10 | 965027+158655725 | find_ele_rec [5]  |
|       |     | 0.08 | 0.01 | 363039/363039    | save_string [8]   |
|       |     | 0.00 | 0.01 | 363039/363039    | new_ele [12]      |
|       |     |      |      | 158655725        | find_ele_rec [5]  |

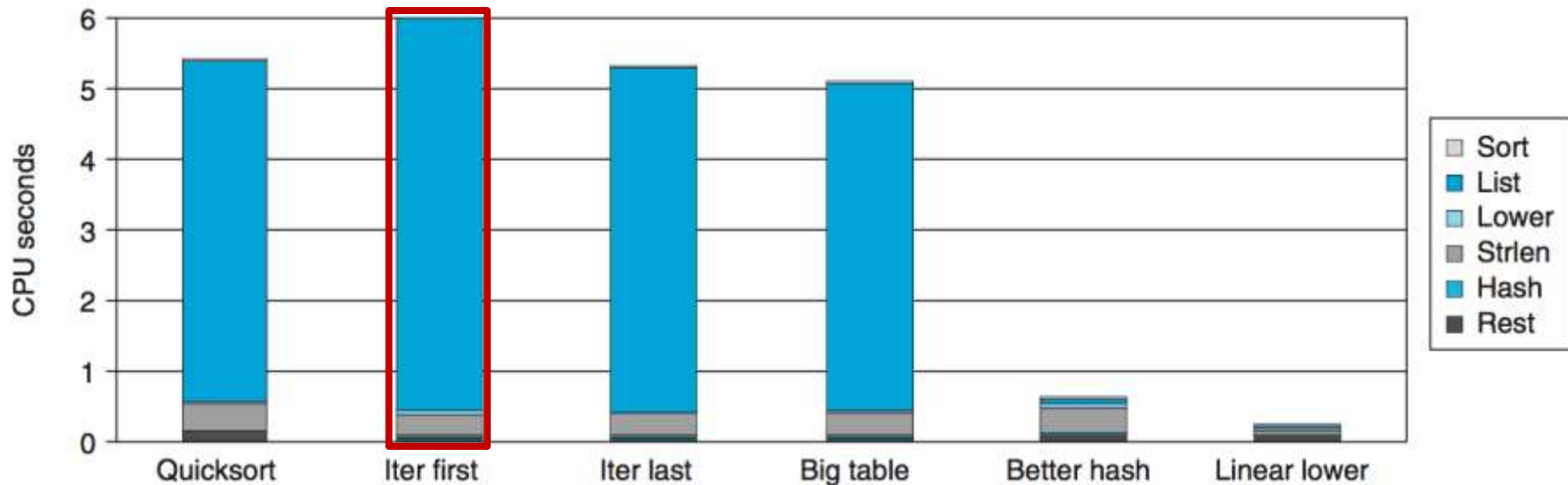
- Ratio :  $158655725/965027 = 164.4$
- The average length of a list in one hash bucket is 164

# Try Code Optimizations



- Replace recursive call to iterative
  - Insert elements in linked list

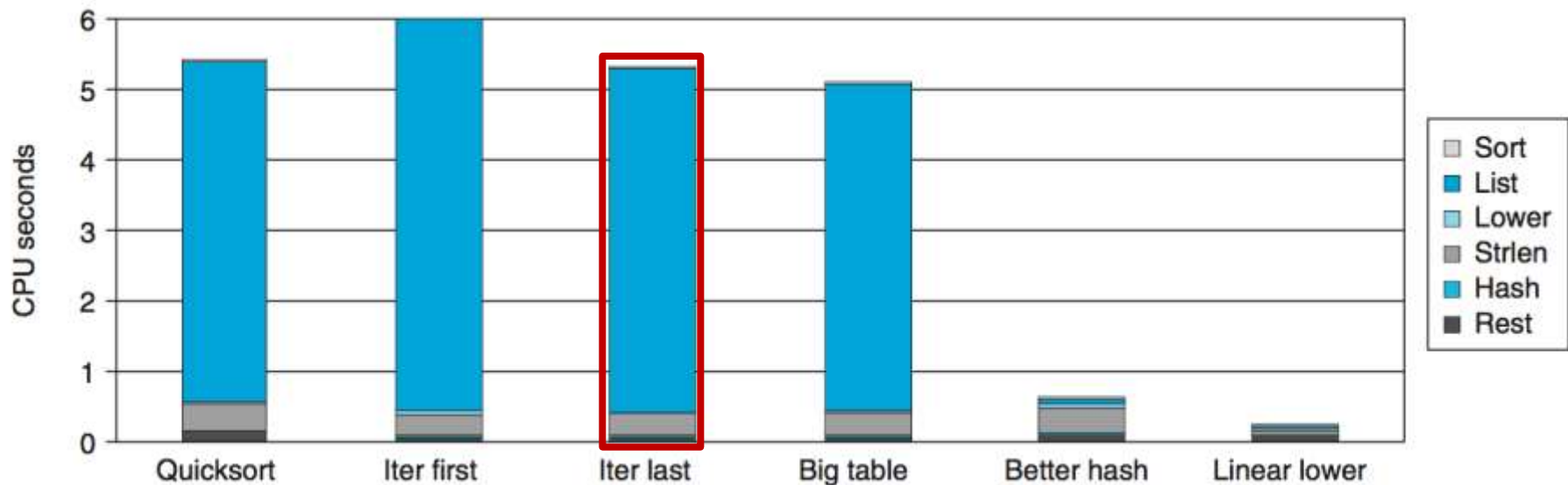
# Try Code Optimizations



- Causes code to slow down
- Reason:
  - Iter first: insert a new element at the beginning of the list
  - Most common n-grams tend to appear at the end of the list which results the searching time

# Try Code Optimizations

---



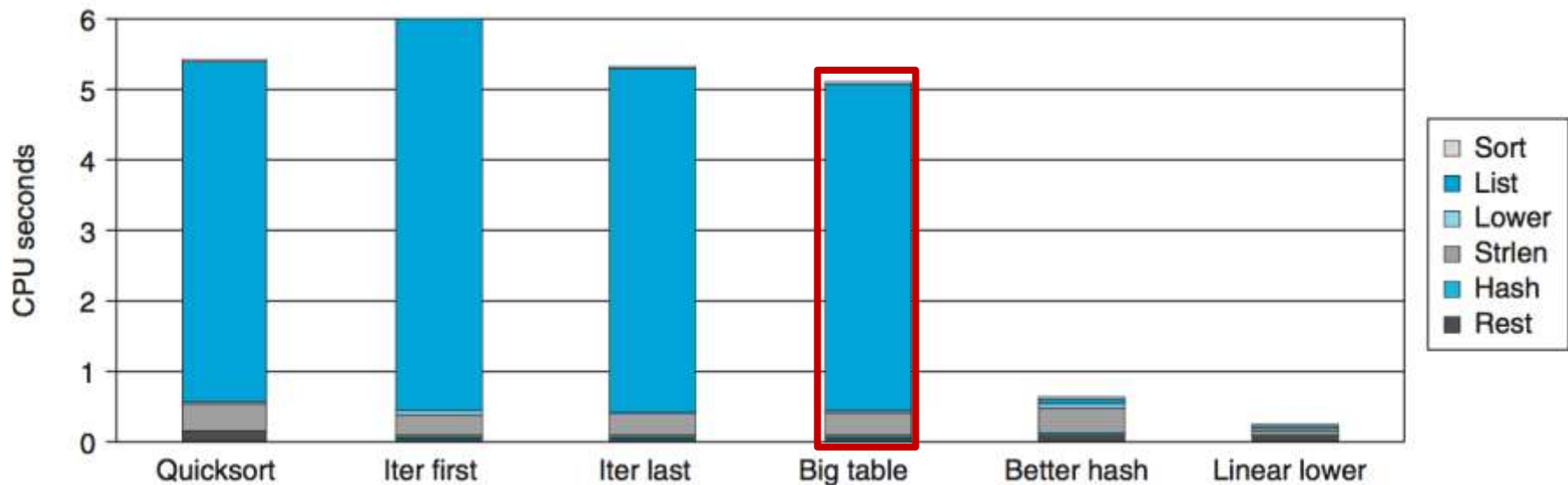
Iter last:

Iterative function, places new entry at end of the list  
Tend to place most common words at front of list



# Try Code Optimizations

---



- Initial version: only 1021 buckets.
- There are  $363039/1021 = 355.6$  bigrams in each bucket
- Big table: Increase it to 199,999

# Try Code Optimizations

---

- Only improves 0.3s

# Try Code Optimizations

---

- Only improves 0.3s
- Reason
  - Hash key is summing character codes for a string
    - The maximum code is 3371
      - for "honorificabilitudinitatibus thou"
    - Cannot differentiate "rat" and "tar"
  - Most buckets are not used

Better hash: Use more sophisticated hash function

Shift and Xor

Time drops to 0.4 seconds

# Try Code Optimizations

---

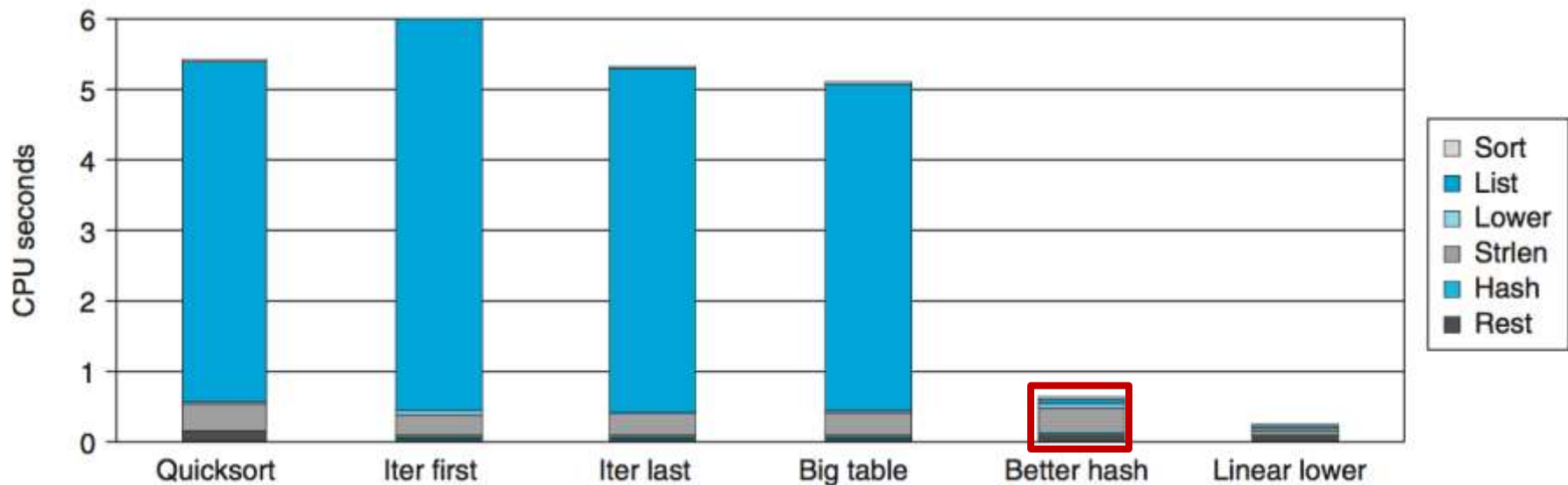
Better hash: Use more sophisticated hash function

- Shift and Xor

- Time drops to 0.4 seconds

# Try Code Optimizations

---



Better hash:

Use more sophisticated hash function (Shift and Xor)

Time drops to 0.4 seconds

# Optimizaitons

---

Strlen takes 0.2 seconds

## Code Motion

---

```
1 /* Convert string to lowercase: slow */
2 void lower1(char *s)
3 {
4     int i;
5
6     for (i = 0; i < strlen(s); i++)
7         if (s[i] >= 'A' && s[i] <= 'Z')
8             s[i] -= ('A' - 'a');
9 }
10
```

# Optimizaitons

---

Strlen takes 0.2 seconds

Linear lower: Move strlen out of loop  
Time drops to 0.2 seconds



## Code Motion

---

```
11 /* Convert string to lowercase: faster */
12 void lower2(char *s)
13 {
14     int i;
15     int len = strlen(s);
16
17     for (i = 0; i < len; i++)
18         if (s[i] >= 'A' && s[i] <= 'Z')
19             s[i] -= ('A' - 'a');
20 }
21
```

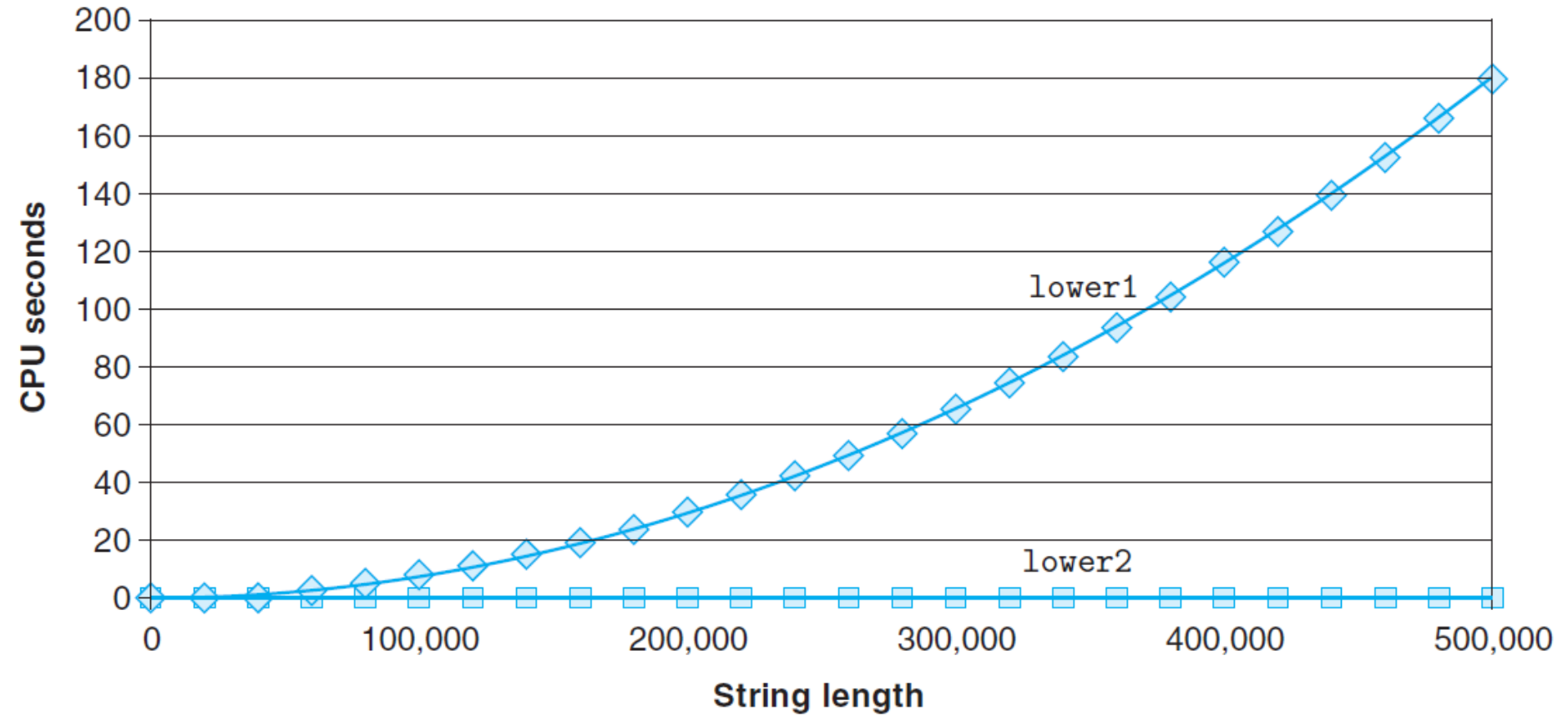
## Code Motion

---

```
22 /* Sample implementation of library function strlen */
23 /* Compute length of string */
24 size_t strlen(const char *s)
25 {
26     int length = 0;
27     while (*s != '\0') {
28         s++;
29         length++;
30     }
31     return length;
32 }
```

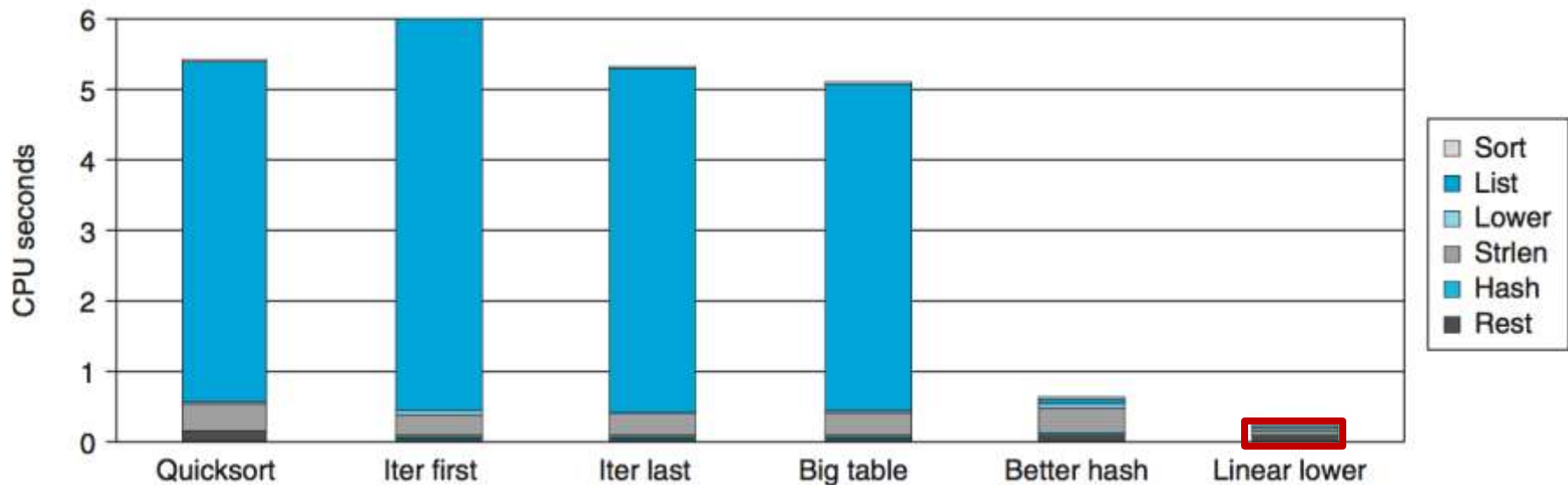
# Code Motion

---



# Further Optimizations

---



Linear lower:

Move strlen out of loop

Time drops to 0.2 seconds

# Performance Tuning

---

- Benefits
  - Helps identify performance bottlenecks
  - Especially useful when have complex system with many components
- Limitations, only shows performance **for data tested**
  - If we had run the original function on data consisting of a smaller number of longer strings, we would have found that the lowercase conversion routine was the major performance bottleneck.
  - Even worse, if it only profiled documents with short words, quadratic inefficiency of `lower1` could remain lurking in code
  - Timing mechanism fairly crude
    - Only works for programs that run for > 1 seconds